

Assessment 3 Lab Report

Foundations of Robotics

RUHAN SHAFI

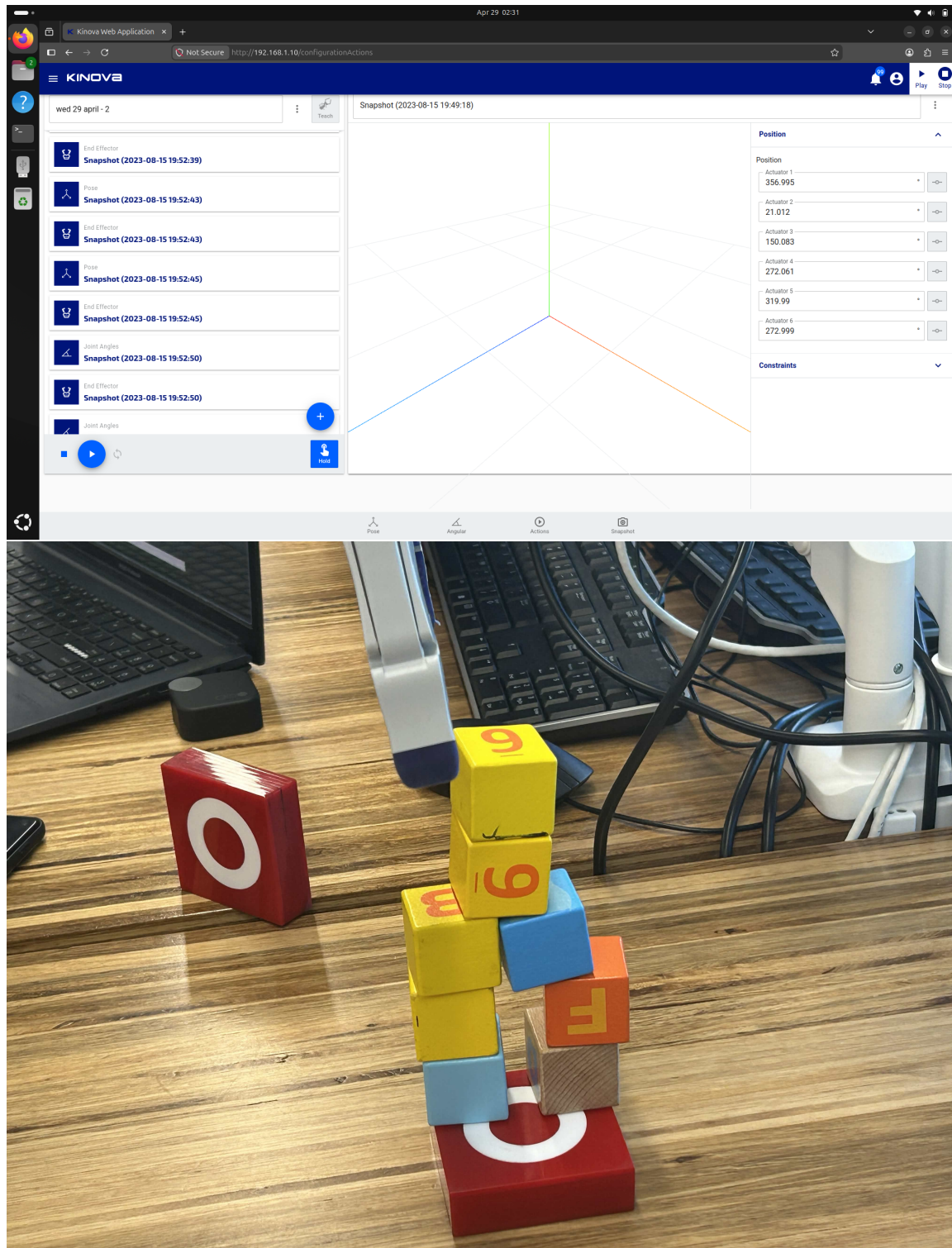


UNIVERSITY OF
CANBERRA

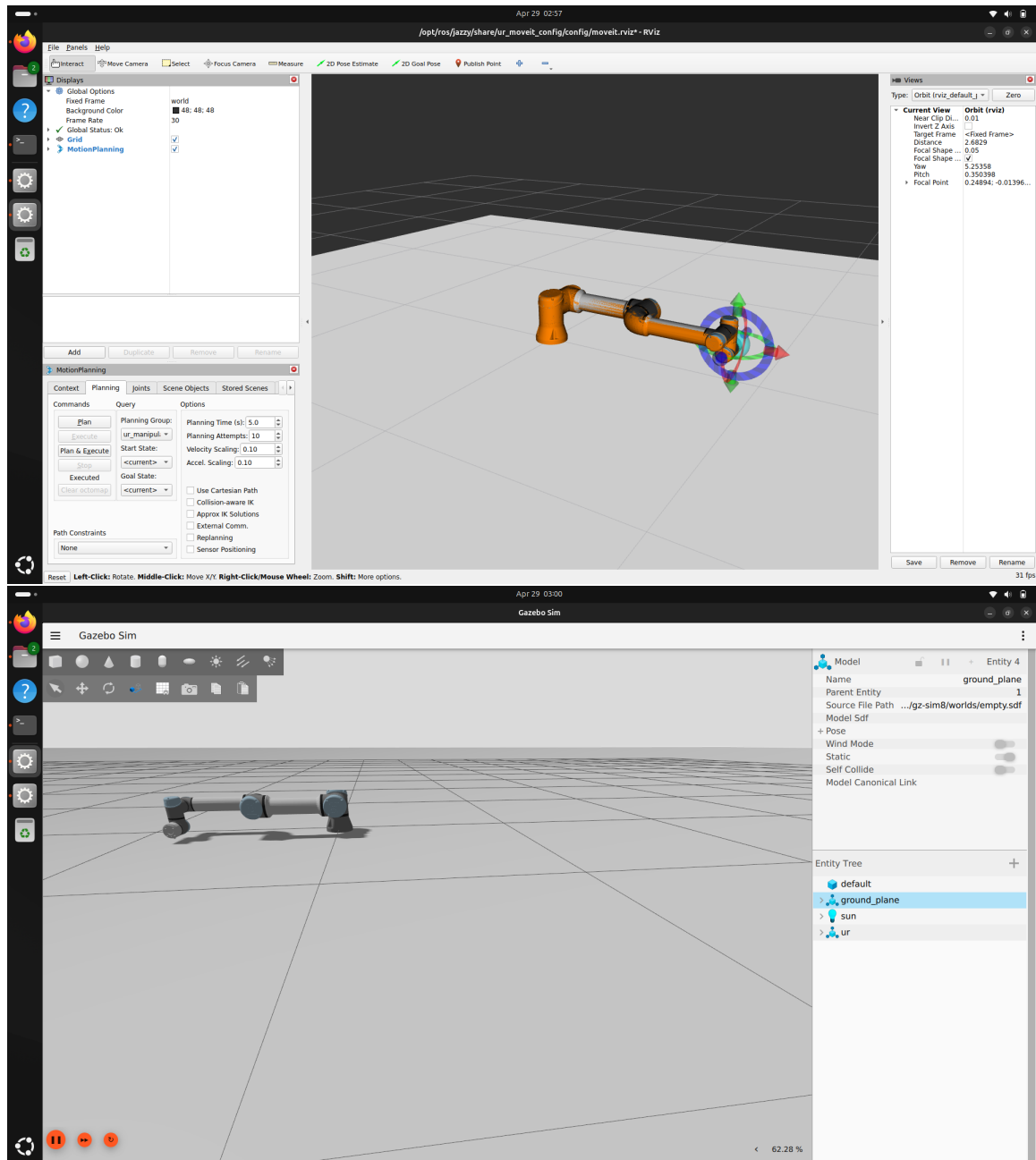
Assessment 3: Lab Report

Task 1

Part 1 - The Build



Part B - The Simulation



Task 2 - Simple Two-Link Manipulator

Part 1 - Manual Working Out

Task 2a For the given Two Link Manipulator (Fig 1),

If $l_1=1\text{m}$ and $l_2=1\text{m}$
 $\theta_1=40^\circ$ and $\theta_2=10^\circ$
Calculate the coordinates of the end effector (x,y) [2 points]
NOTE: Please provide complete solution, indicating how you arrived at the result.

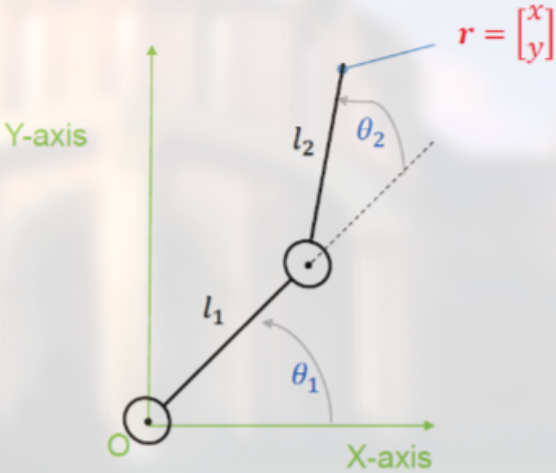


Fig 1: Two Link Manipulator

Given the Two-Link Manipulator in Fig 1, with the following parameters:

- $l_1 = 1\text{ m}$
- $l_2 = 1\text{ m}$
- $\theta_1 = 40^\circ$
- $\theta_2 = 10^\circ$

The coordinates (x,y) of the end effector are determined by the sum of the link vectors. Note that the angle of the second link relative to the x-axis is $(\theta_1 + \theta_2)$.

$$\begin{aligned}x &= l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \\y &= l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2)\end{aligned}$$

Substitute the given values into the equations:

$$\begin{aligned}x &= 1 \cdot \cos(40^\circ) + 1 \cdot \cos(40^\circ + 10^\circ) \\x &= \cos(40^\circ) + \cos(50^\circ) \\x &\approx 0.7660 + 0.6428 \\x &\approx \mathbf{1.4088\text{ m}}\end{aligned}$$

$$\begin{aligned}y &= 1 \cdot \sin(40^\circ) + 1 \cdot \sin(40^\circ + 10^\circ) \\y &= \sin(40^\circ) + \sin(50^\circ) \\y &\approx 0.6428 + 0.7660 \\y &\approx \mathbf{1.4088\text{ m}}\end{aligned}$$

The coordinates of the end effector $\mathbf{r}$ are:

$$\mathbf{r} = \begin{bmatrix} x \\ y \end{bmatrix} \approx \begin{bmatrix} 1.4088 \\ 1.4088 \end{bmatrix} \text{ m}$$

Part B - Pseudo Code

Write a suitable algorithm to calculate the Forward Kinematics of a 2-link manipulator robot with link lengths of l_1 , l_2 and joint angles θ_1 , θ_2 respectively.

```
1 // Inputs:  l1, l2      - link lengths in metres
2 //           theta1     - joint 1 angle in degrees (measured from X-axis)
3 //           theta2     - joint 2 angle in degrees (measured relative to link 1)
4 // Outputs: x, y        - end effector position in metres
5 // Constant: PI ← 3.14159265
6
7 METHOD ForwardKinematics(l1, l2, theta1, theta2)
8
9     // Step 1: Convert joint angles from degrees to radians
10    theta1_rad ← theta1 * (PI / 180)
11    theta2_rad ← theta2 * (PI / 180)
12
13    // Step 2: Calculate the sum of joint angles
14    // This gives the absolute angle of link 2 relative to the X-axis
15    theta_sum ← theta1_rad + theta2_rad
16
17    // Step 3: Calculate end effector x coordinate
18    // x = l1*cos(theta1) + l2*cos(theta1 + theta2)
19    x ← (l1 * COS(theta1_rad)) + (l2 * COS(theta_sum))
20
21    // Step 4: Calculate end effector y coordinate
22    // y = l1*sin(theta1) + l2*sin(theta1 + theta2)
23    y ← (l1 * SIN(theta1_rad)) + (l2 * SIN(theta_sum))
24
25    // Step 5: Return the end effector position vector
26    PRINT "End Effector Position:"
27    PRINT "x = " + x
28    PRINT "y = " + y
29
30    RETURN x, y
31
32 ENDMETHOD
33
34 // --- Main Program ---
35
36 READ l1
37 READ l2
38 READ theta1
39 READ theta2
40
41 CALL ForwardKinematics(l1, l2, theta1, theta2) RETURNING x, y
```

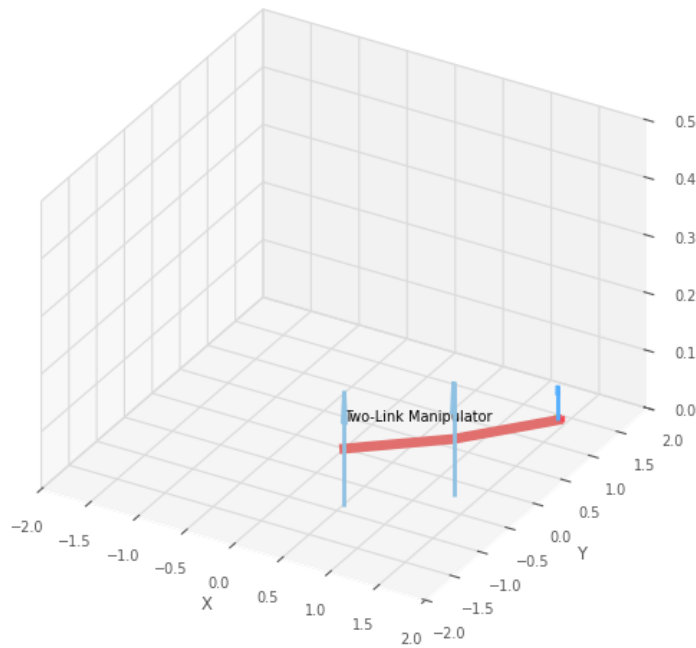
Task 3 - Forward Kinematics

Task 3a - Python Script

Screenshot of the resulting DH parameter table and graphical model of the robot configuration.

DHRobot: Two-Link Manipulator, 2 joints (RR), dynamics, standard DH parameters

θ_i	d_i	a_i	α_i
q1	0	1	0.0°
q2	0	1	0.0°



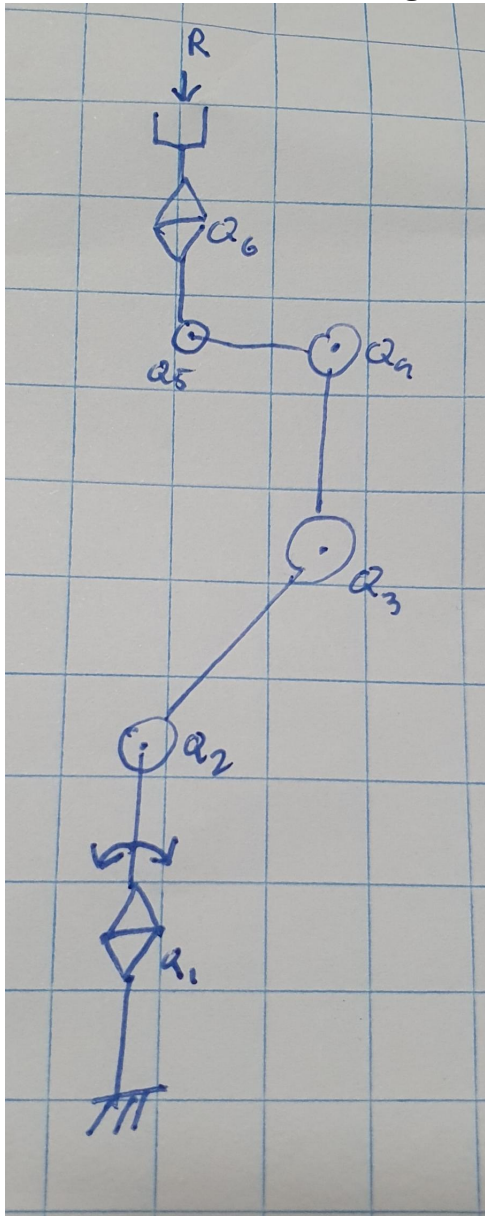
[1.40883205 1.40883205 0.]
<Figure size 640x480 with 0 Axes>

Record the output and compare the results to what you calculated in Task 2a

As seen in the screenshot above, the returned value is the same as the ones calculated in Task 2a, thus validating the calculations performed in that task.

Task 3b - DH Parameters

Task 1 - Draw the Link diagram



Task 2 - Tabulate the DH parameters

$${}^{i-1}T_i = \begin{bmatrix} \cos(\theta_i) & -\cos(\alpha_i)\sin(\theta_i) & \sin(\alpha_i)\sin(\theta_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\alpha_i)\cos(\theta_i) & -\sin(\alpha_i)\cos(\theta_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 63: Classical DH parameters frame definitions

The following table gives the Classical DH parameters for the robot.

Table 50: Classical DH parameters

i	α_i	a_i	d_i	θ_i
1	$\pi/2$	0.0	(128.3 + 115.0)	q_1
2	π	280.0	30.0	$q_2 + \pi/2$
3	$\pi/2$	0.0	20.0	$q_3 + \pi/2$
4	$\pi/2$	0.0	(140.0 + 105.0)	$q_4 + \pi/2$
5	$\pi/2$	0.0	(28.5 + 28.5)	$q_5 + \pi$
6	0	0.0	(105.0 + 130.0)	$q_6 + \pi/2$

Here q_i refers to the joint angles.

The DH parameters use the following frames definitions:

Sourced from the Official Kinova Gen3 lite robot User guide: Page 135

Task 3c- Python Script

Source code to generate the Kinova robot model using the Robotics Toolbox

```

1 import math
2 import roboticstoolbox as rtb # load the robotics toolbox
3
4 # the robot model can be defined using the the robotics toolbox
5 # The DHRobot object stores the joint information for the robot
6 # RevoluteDH objects can be used to represent each revolute joint in the robot model
7 # DH parameters are used to define the robot joint relationships
8 q = [math.radians(357.016), math.radians(21.016), math.radians(150.082), math.radians(271.999),
9       math.radians(319.967), math.radians(272.969)]
10
11 robot_2dof = rtb.DHRobot(
12     [
13         # the RevoluteDH class represents a revolute joint using DH parameters
14         # this is a list of 2 joints representing 2 degrees of freedom (DOF)
15         rtb.RevoluteDH(

```



```

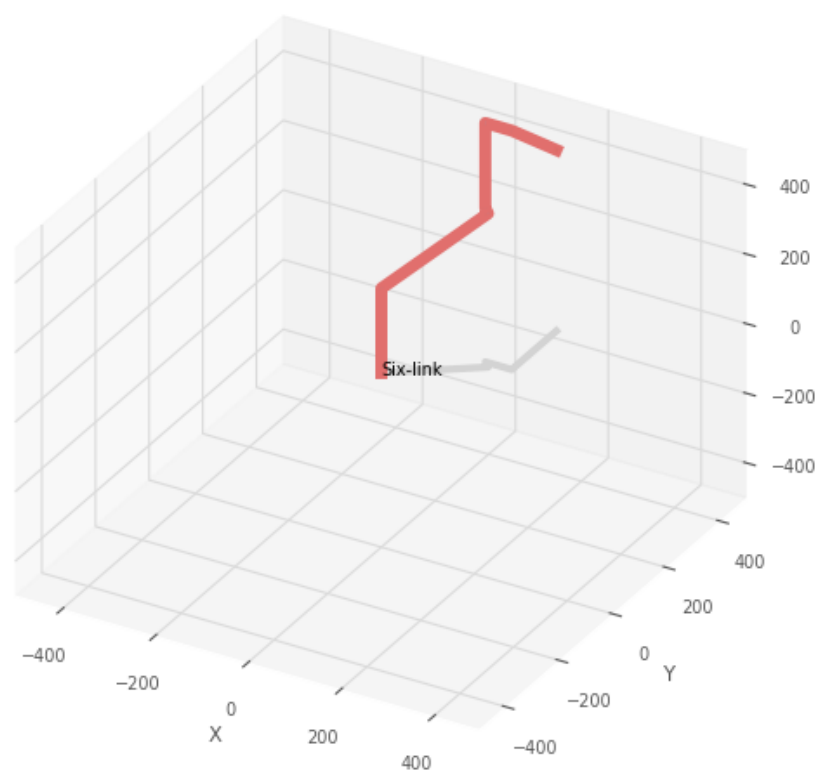
15         d = (128.3+115.0),
16         alpha = (math.pi/2),
17         a = 0, # note that the dimensions are in m instead of mm
18         offset = 0
19     ),
20     rtb.RevoluteDH(
21         d = 30.0,
22         alpha = (math.pi),
23         a = 280,
24         offset = (math.pi/2)
25     ),
26     rtb.RevoluteDH(
27         d = 20,
28         alpha = (math.pi/2),
29         a = 0, # note that the dimensions are in m instead of mm
30         offset =(math.pi/2)
31     ),
32     rtb.RevoluteDH(
33         d = (140.0+105.0),
34         alpha = (math.pi/2),
35         a = 0,
36         offset =(math.pi/2)
37     ),
38     rtb.RevoluteDH(
39         d = (28.5+28.5),
40         alpha = (math.pi/2),
41         a = 0, # note that the dimensions are in m instead of mm
42         offset =math.pi
43     ),
44     rtb.RevoluteDH(
45         d = (105.0+130.0),
46         alpha = 0,
47         a = 0,
48         offset =(math.pi/2)
49     )
50 ],
51 name = "Six-link"
52 )
53
54 # this will print the robot model details
55 print(robot_2dof)
56
57 # this will plot the robot model graphically
58 #robot_2dof.plot(q, limits=[-2.0, 2.0, -2.0, 2.0, 0.0, 0.5])
59
60 '''
61 Find the end effector pose given the measured joint angles
62 '''
63
64 # create an array of joint angles
65 # for example:
66 #
67 # joint_angles = np.array([0,0,0,0,0,0])
68
69 # plot the robot model
70 robot_2dof.plot(q, limits=[-500, 500, -500, 500, -500, 500])
71
72 # get the end effector pose
73 pose_2 = robot_2dof.fkine(q)
74
75 # print the end effector pose
76 print(pose_2.t)

```

Screenshot of final pose of the robot as generated by the robotics toolbox and output of the command `print(pose.t)`

θ_i	d_i	a_i	α_i
q1	243.3	0	90.0°
q2	30	280	180.0°
q3	20	0	90.0°
q4	245	0	90.0°
q5	57	0	90.0°
q6	235	0	0.0°

--	--



[204.07106781 299.09902577 520.11980515]
<Figure size 640x480 with 0 Axes>